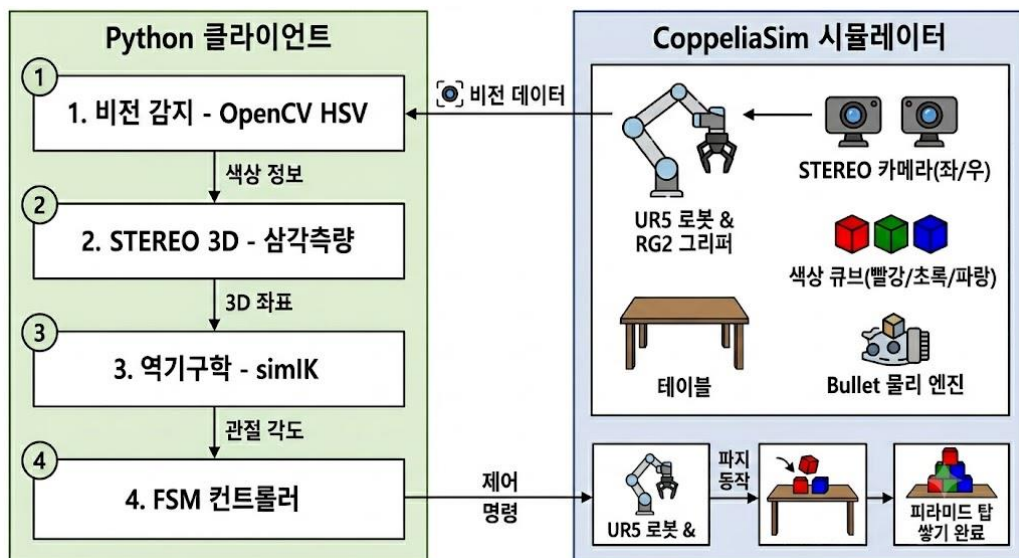


Stereo Vision 기반 UR5 로봇의 3차원 피라미드 탑 쌓기

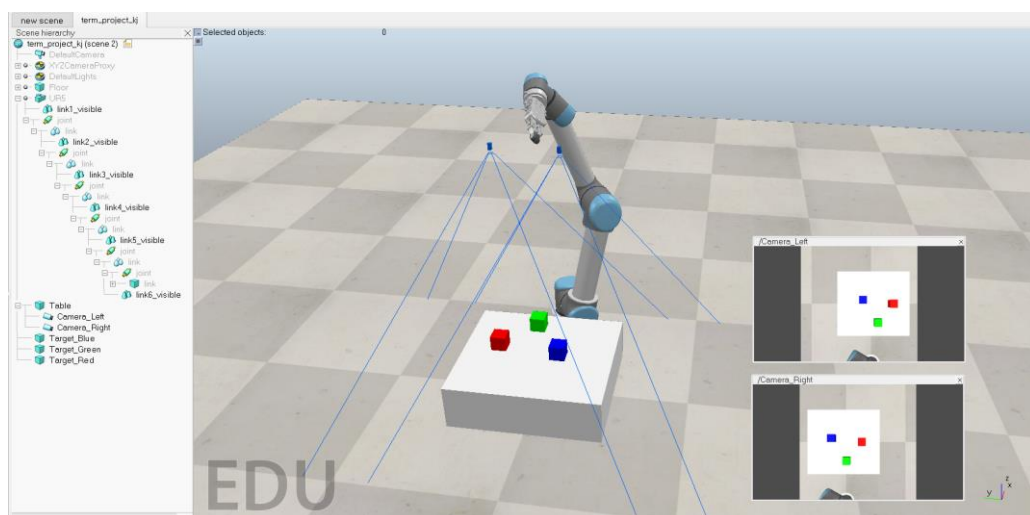
213157 박관준

1. 프로젝트 개요

- CoppeliaSim 내에서 UR5 로봇이 3개의 색상 블록(Red · Green · Blue)을 인식하여 순차적으로 피라미드 형태의 탑을 쌓는 자동화 시스템 구현
- CoppeliaSim(시뮬레이터·서버) ↔ Python(제어·클라이언트) 간 ZMQ Remote API 통신
- 핵심 기술: Stereo Vision(삼각측량), 역기구학(simIK), 유한상태기계(FSM)



시스템 아키텍처



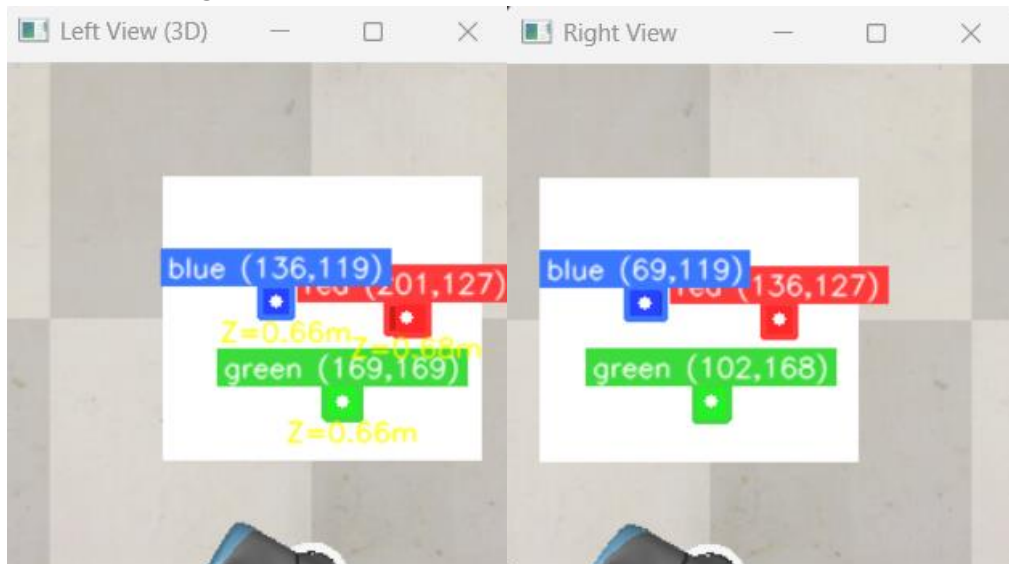
전체 Scene 구성

2. 비전 인식 (2D 색상 검출)

각 블록의 이미지 중심 픽셀 좌표 (u, v)를 색상 기반으로 추출한다.

(vision_test.py)

- 처리 파이프라인
 - ① getVisionSensorImg로 카메라 영상 획득
 - ② 영상 보정 - CoppeliaSim 영상은 상하 반전 + RGB 포맷
→ cv2.flip + RGB2BGR
 - ③ HSV 변환 후 색상 마스크링 - Red/Green/Blue 각각 inRange
- 시각화 (Segmentation)
검출 영역을 반투명 색으로 칠하고(addWeighted) 윤곽선 + 라벨 박스를 그려 instance segmentation 형태로 표현.



vision_test.py 실행결과

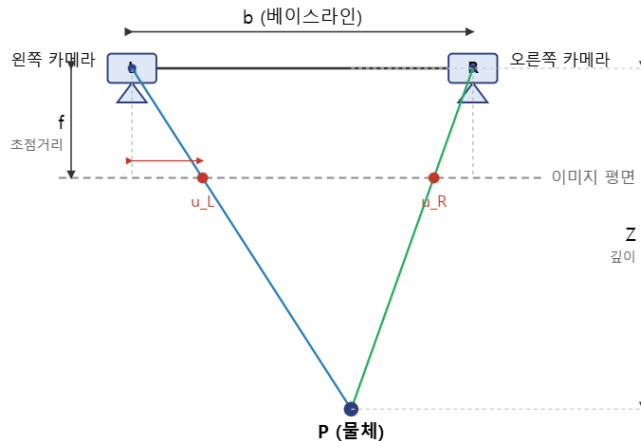
3. Stereo Vision 및 3D 타겟팅

2D 픽셀 좌표를 Epipolar 기하학 + 삼각측량으로 3D 월드 좌표로 변환한다.

(stereo_test.py)

- 카메라 구성
 - 좌·우 카메라는 책상 위에 배치되며 월드 Y축으로만 베이스라인
 $b = 0.2\text{m}$ 만큼 떨어져 있음
 - 카메라 회전을 분석한 결과 이미지 가로축(u)이 월드 Y축과 정렬
→ 시차가 가로 방향으로 생기는 표준 형태

- 삼각측량



Stereo 삼각측량

- 내부 파라미터: FOV로부터 초점거리 $f(px)$ 와 주점 (C_x, C_y) 계산
- 시차: $d = |u_L - u_R|$
- 깊이: $z = \frac{f \cdot b}{d}$
- 카메라 기준 3D: $X_c = \frac{(u_L - C_x)Z}{f}, Y_c = \frac{(u_R - C_y)Z}{f}$
- 월드 변환: 카메라 pose(R, C)를 sim.getObjectMatrix로 런타임 조회하여
 $P_{world} = RP_{cam} + C$
- 결과

3개 블록 모두 실제 위치 대비 오차 1cm 내외의 정확도 달성.

```
PS D:\Workspace\Robot_term_project> python stereo_test.py
```

color	camera (Xc,Yc,Zc)	world (X,Y,Z)	err(cm)
red	(-0.224,+0.003,+0.672)	(-0.503,+0.124,+0.178)	1.2
green	(-0.122,-0.122,+0.662)	(-0.378,+0.022,+0.188)	0.4
blue	(-0.024,+0.027,+0.662)	(-0.527,-0.076,+0.188)	0.3

stereo_test.py 실행결과

4. 로봇 기구학(Kinematics) 솔버

추정한 월드 좌표로 로봇 엔드이펙터(TCP)를 이동시킨다. CoppeliaSim의 simIK 모듈을 활용한다. (kinematics_test.py)

- DH 파라미터·정기구학을 직접 구현하는 대신, simIK가 base→tip 운동 사슬을 자동 구성하고 IK를 풀이
- 베이스(UR5)가 월드 원점에 있어 베이스 좌표 변환이 불필요 → 월드 좌표를 그대로 목표로 사용 (큰 단순화)
- IK 해는 IK 환경에서 풀고, 실제 로봇에는 setJointTargetPosition으로 적용
 → 동역학(물리) 컨트롤러와 충돌 없이 부드럽게 이동

```

ik_env = simIK.createEnvironment()
ik_group = simIK.createGroup(ik_env)
simIK.setGroupCalculation(ik_env, ik_group,
                           simIK.method_damped_least_squares, 0.3, 99)
_, sim_to_ik, _ = simIK.addElementFromScene(
    ik_env, ik_group, base, tip, target, simIK.constraint_position)

```

위치제약 IK 구성

5. 작업 시퀀스 제어 (FSM)

탑 쌓기 알고리즘을 유한상태기계로 구현한다. (stacking_test.py)

5.1 상태기계

각 블록마다 다음 상태를 순차 진행한다.

- 접근(위) → 하강 → 파지 → 상승 → 이동 → 하강(배치) → 놓기 → 복귀
- APPROACH_PICK → DESCEND_PICK → GRASP → LIFT →
APPROACH_PLACE → DESCEND_PLACE → RELEASE → RETREAT

5.2 'ㄷ'자 충돌회피 귀적

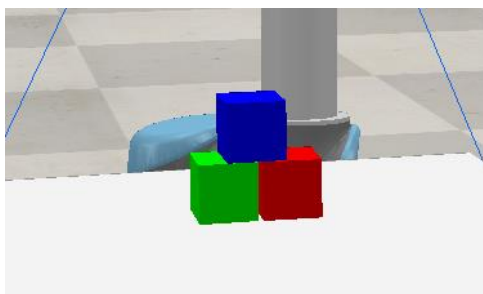
항상 안전 고도(SAFE_Z)까지 올린 뒤 수평 이동 → 하강하여 다른 블록·기존 탑과의 충돌을 회피.

5.3 그리퍼 제어

- RG2 그리퍼는 RG2_open 신호로 개폐 (1=열림, 0=닫힘)
- 파지 자세: 위치 IK에 자세 제약을 추가해 그리퍼 방향을 고정
- GRIP_TILT(approach 축 기울기) + GRIP_ROLL(손가락 닫히는 평면)으로 일정한 자세 유지
- approach 축을 비스듬히 기울여 작업영역 가장자리의 특이점(팔이 빙빙도는 현상)을 회피

5.4 피라미드 기하

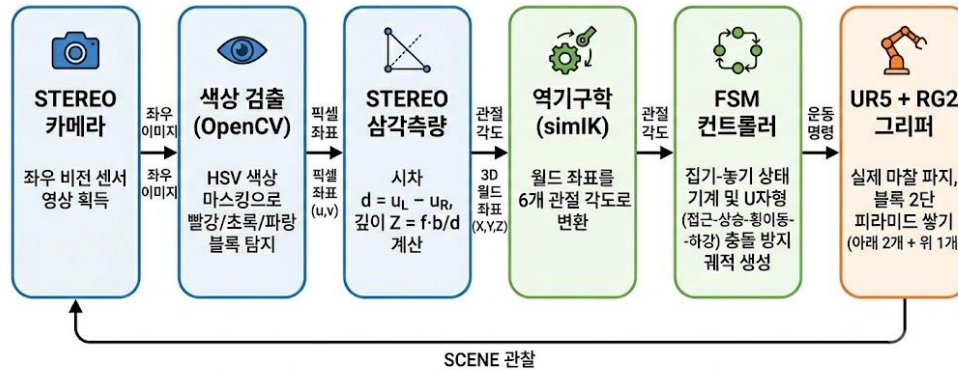
- 블록 한 번 길이 기준으로 바닥 2개를 좌우로 배치하고, 꼭대기 1개를 그 위 중앙에 배치
- 파지 오프셋을 고려해 배치 높이를 보정하고, 목표보다 살짝 위에서 놓아 기존 블록을 누르지 않도록 함



완성된 피라미드

6. 전체 시스템 통합

비전 인식과 탐 썬기를 하나로 연결하여 "카메라로 보고 → 실제로 집어서 썬기" 전체 파이프라인을 완성한다. (realgrasp_pipeline.py)



전체 파이프라인

- 검출은 여러 프레임을 평균하여 노이즈를 완화
- 색상은 고유하므로 검출 색 → 블록 핸들 매핑으로 파지 대상을 특정
- 픽 위치를 썬 좌표가 아닌 비전 검출 좌표로 사용 → 진정한 인식 기반 동작
- 비전 정확도는 1cm 내외로, 통합 후에도 안정적으로 동작

7. 결론 및 시행착오

그리퍼를 실제로 달아 마찰력으로 블록을 집어 든다. 실제 물리 파지는 마찰·힘·질량·형상이 모두 맞아야 성공하는 민감한 문제로, 다음 요소들을 조정하여 안정적인 파지를 달성했다.

- 객체의 dynamic 속성 - 블록이 static이면 물리적으로 움직이지 않아 파지 불가 → "body is dynamic" 활성화 필수
- 마찰계수 - 마찰이 0이면 아무리 힘이 세도 미끄러짐 → 블록·그리퍼 양쪽 접촉면 마찰을 충분히 부여
- 파지 깊이 (GRASP_Z_OFFSET) - 너무 깊으면 블록을 튕겨내고, 너무 얕으면 헛집음 → 적절한 높이 튜닝
- 블록 형상 — 원통은 둥근 면 때문에 미끄러지기 쉬워, 정육면체로 변경하여 평평한 면을 안정적으로 파지
- 그리퍼 회전 (GRIP_ROLL) - 손가락 닫히는 축을 블록 배치 축과 수직으로 두어, 배치 시 이웃 블록을 밀지 않도록 조정
- 자세 보간 (slerp) - 이동 시 손목 자세를 한 번에 스냅하지 않고 구면 선형 보간하여 초기 급격한 회전(특이점) 방지